

Starter Project #3: Code Review Report

For this assignment, I worked on improving and reviewing a Boggle Solver project written in Python. The purpose of the assignment was not only to create a functioning Boggle solver, but also to practice professional software engineering workflows such as peer code reviews, static analysis, debugging, and regression testing. The program takes a Boggle board and a dictionary of words and searches for all valid words that can be formed using depth-first search (DFS) and backtracking techniques. My overall approach was to first focus on functionality and correctness before improving readability, organization, and code quality. Once the solver was working properly, I focused on implementing feedback from my partner, improving testing coverage, and fixing all linting and formatting issues reported by static analysis tools.

One of the major parts of this assignment was the peer review process. The feedback I received from my partner focused mainly on readability, organization, and Python best practices. My partner mentioned that using a class for the Boggle solver was a strong design choice because it kept the code organized and easier to maintain. They also liked that I implemented prefix pruning and used fast lookup structures for searching words efficiently. Another important point of feedback was improving validation and edge-case handling. My partner suggested making sure the program safely handled invalid or empty inputs rather than potentially crashing. This encouraged me to improve my validation methods for both the grid and dictionary inputs.

I also received feedback related to making the code more Pythonic and easier to understand. For example, my partner suggested using `isinstance()` instead of checking data types with `type()`. They also recommended replacing hardcoded values with constants, such as defining a minimum word length constant instead of repeatedly using the number three directly in the code. Other suggestions focused on readability improvements, including adding more descriptive comments, expanding docstrings, and explaining how prefix pruning improves the performance of the DFS algorithm. My partner additionally suggested defining neighbor directions only once instead of recreating them repeatedly during recursive searches. Another small but useful recommendation was renaming certain variables to make their purpose clearer and more professional.

While reviewing my partner's code, I focused mainly on organization, readability, and testing coverage. One issue I noticed was that some functions lacked comments or explanations, which made it difficult to immediately understand the purpose of certain sections of code. I recommended adding docstrings and inline comments, especially around more complicated logic such as recursion or search algorithms. I also suggested using more descriptive variable names because some abbreviated names reduced readability. Another issue I identified was inconsistent formatting and indentation. Since Python depends heavily on indentation, I emphasized the importance of using consistent spacing and avoiding mixing tabs with spaces. I also suggested

Paris Alston

CSCI 375

Professor Burge

running a linter earlier in development because it is much easier to fix formatting issues gradually rather than all at once near the end of a project. Finally, I recommended adding more edge-case tests, including empty grids, invalid dictionaries, repeated tile usage, and special tile combinations such as “Qu” and “St.”

After reviewing the feedback, I implemented several major improvements to my project. One of the biggest changes was replacing dictionaries used for lookups with sets because sets provide faster and cleaner membership checking. I also added constants such as `MIN_WORD_LENGTH` and a predefined `DIRECTIONS` list to eliminate hardcoded values and improve readability. Another major improvement was restructuring the DFS implementation to make the recursion cleaner and easier to follow. I added additional comments explaining the prefix-pruning optimization and why it improves performance by preventing the algorithm from exploring impossible paths. I also improved the validation methods by replacing `type()` checks with `isinstance()` and expanding the docstrings throughout the project to make the code easier to understand.

In addition to improving the solver itself, I also spent significant time improving the overall code quality and formatting. Initially, the project had many PEP8 and static analysis violations. Some of the issues included missing whitespace after commas, incorrect indentation, trailing whitespace, mixed tabs and spaces, semicolons at the ends of lines, overly long lines, improperly formatted comments, and missing newlines at the ends of files. I carefully fixed each issue and reran the linter multiple times until all errors were resolved. Although the number of formatting errors initially felt overwhelming, the process helped me gain a stronger understanding of Python style standards and why consistency is important in collaborative software development environments.

After implementing all improvements, I used regression testing to verify that the program still functioned correctly. I reran all existing test cases after every major modification to ensure that the changes did not introduce new bugs or break existing functionality. The test cases included diagonal adjacency checks, no tile reuse validation, special tile handling for “Qu” and “St,” empty grid cases, and small board edge cases. I also verified that sorting behavior and case normalization continued to work properly after refactoring sections of the code. Running repeated tests throughout development helped confirm that the improvements to readability and performance did not negatively impact the correctness of the solver.

Overall, this assignment taught me a great deal about the importance of code reviews and professional development workflows. Before this project, I mainly focused on whether my code simply produced the correct output. Through the review process, I learned that readability, maintainability, formatting, testing, and collaboration are just as important as functionality. Receiving feedback from another person helped me notice issues that I would not have caught on my own, especially related to organization and coding style. At the same time, reviewing

Paris Alston

CSCI 375

Professor Burge

someone else's code improved my ability to identify weaknesses and think critically about software quality. I also gained much more experience using static analysis tools and debugging PEP8 errors. Overall, this project helped me better understand how professional software engineering involves continuous improvement, collaboration, testing, and writing clean, maintainable code rather than simply making a program work.